

MedFAQ

A medical FAQ chatbot built from scratch

This document explains the project in detail. It covers what the project does, how each part works, how the parts connect, and how the models were built and trained. Language is kept plain and short. No machine learning library was used as a black box. Every model in this project was written by us in numpy or in PyTorch using only tensors and autograd.

Project	MedFAQ
Team	Abdul Ghani, Anas Bhatti, Muhammad Salman, plus 2 members
University	NUTECH, 6th semester
Repository	github.com/AbdulGhani002/medical-faq-auto
Stack	FastAPI, Next.js, MongoDB, Docker, numpy, PyTorch
LLM used	None. Every model was trained from scratch on our corpus.

1. What the project is

MedFAQ is a chatbot that answers patient questions about three medical specialties: Radiology, Physiotherapy, and Cardiology. The chatbot does two things at once.

- It answers the user with a machine written reply that is grounded in a real, clinician written FAQ.
- It learns from chat traffic. When many users ask the same kind of question, the system clusters those questions and proposes a new FAQ for the clinician to review.

The chatbot never invents medical facts. The text it generates is checked against the closest matching FAQ. If the generated text drifts away from the FAQ, the system falls back to the FAQ word for word. This keeps the answers safe.

2. Folder layout

The repository has five top level folders. Each one has a clear job.

Folder	Role	What is inside
api/	Backend service	FastAPI app, every model, every NLP module, training scripts.
web/	Frontend	Next.js 15 site for chat, playground, admin, architecture pages.
pipeline/	Offline pipeline	Chat log to FAQ candidate mining script that can run nightly.
data/	Corpus	All JSON and JSONL data files plus expansion and augmentation scripts.
docs/	Generated docs	PDF reports and supporting documentation.

Inside **api/app/** there are three important folders:

- **mym/** contains every machine learning model we wrote. Plain numpy or plain PyTorch. No sklearn, no rank_bm25, no sentence transformers.
- **nlp/** contains the language tools that sit on top of those models, such as the lemmatiser, the POS tagger, the NER tagger, the sentiment lexicon, and the dialog manager.
- **services/** contains the glue. It loads the models, runs the retrieval, calls the generator, talks to MongoDB, and runs the auto FAQ pipeline.

3. How one chat turn works

Every time a user sends a message, the same pipeline runs. The pipeline has ten short steps. Each step is a small Python function. The whole thing finishes in about 50 to 100 ms on a normal laptop CPU.

Step 1: PHI scrub. Patient health information like phone numbers, ID cards, emails, and dates are masked. This runs before anything is logged.

Step 2: Roman Urdu expand. If the user writes in Roman Urdu, common words like 'sar', 'dard', 'seenay' are mapped to English equivalents so the retriever can match them.

Step 3: Coreference. If the user writes 'how do I treat it', we replace 'it' with the last topic of the chat so the retriever knows what the user means.

Step 4: Context augment. If the message is short and looks like a follow up, we glue the previous user message to the front for more context.

Step 5: Spell correct. Each word is checked against the FAQ vocabulary using Damerau Levenshtein edit distance. Typos are fixed to the closest known word.

Step 6: Query understanding. We tag the message with POS, run NER, detect negation, score sentiment, check for emergency words, and classify the user intent.

Step 7: 5 channel retrieval. We score every FAQ in the specialty against the user message using five different methods at once: TF IDF on words, TF IDF on character n grams, BM25 Okapi, LSA topic similarity, and PPMI word embeddings.

Step 8: Rerank and diversify. The blended top results are reranked with a logistic regression. Then MMR removes near duplicate alternatives so the 'Did you mean' list is varied.

Step 9: Machine writing. The seq2seq generator writes an answer for the user. We compare the generated text to the top FAQ. If the two agree, we send the generated text. If they disagree, we send the FAQ text instead and tell the admin panel.

Step 10: Dialog and counters. The dialog manager adds an opener if the user sounds upset, an emergency banner if the message has red flag words, and a 'Did you mean' card if the top score is close to the next one. The FAQ count is increased by 1 in MongoDB so popular questions float to the top of the public FAQ list.

4. The data

The corpus has 17 files. All of them are stored as JSON or JSONL so they can be edited by hand and streamed by code. The size on disk is about 9 MB. The corpus is grown by three expansion scripts in **data/**.

File	Records	What it holds
faqs.jsonl	214	Clinician written FAQs across the three specialties
intents.jsonl	387	Hand written intent training examples
intents_augmented.jsonl	1517	6 way augmentation of the curated intents
eval_queries.jsonl	156	Held out queries with expected keywords
ner_examples.jsonl	32	BIO tagged sentences for the CRF NER
kg_triples.jsonl	634	Auto mined subject verb object triples
sentiment_lexicon.jsonl	96	Polarity weights for sentiment analysis
synth_chat_logs.jsonl	14357	Synthetic patient chat sessions for training
chat_logs.jsonl	11 to 18	Hand picked chat sessions for the pipeline

The augmentation rules are stored in **data/augment_intents.py**. Each curated intent is rewritten six different ways: synonym swap, phrase swap, prefix variant, casing change, single typo, and prefix plus suffix wrap. The output is deduplicated so we do not feed the same example twice.

The synthetic chat logs are generated by **data/synth_chat_logs.py**. Each FAQ is wrapped in a short session that looks like a real patient asking the question in their own words. About 55 percent of the sessions also have a natural follow up turn, like 'and what about water?'.

5. The models

Every model in MedFAQ was written by us. No pretrained weights are downloaded. The classical models are in numpy. The two trainable neural models are in PyTorch and use only tensors and autograd. They train on a normal laptop CPU and finish much faster on a GPU.

5.1 Classical models in numpy (api/app/myml/)

Module	What it is	How it works
tfidf.py	TF IDF vectoriser	Counts terms, smooths IDF, scales by log frequency, L2 normalises.
bm25.py	BM25 Okapi	Lucene style BM25 with length normalisation.
nb.py	Naive Bayes	Multinomial NB with Laplace smoothing and stable log sum exp.
logreg.py	Logistic Regression	Multinomial softmax with batch gradient descent and L2.
svd.py	Truncated SVD	Randomised SVD by Halko et al for LSA.
kmeans.py	K Means	K means++ initialisation and Lloyd iterations.
mlp.py	MLP	Forward and backward pass, ReLU and softmax, mini batch SGD.
attention.py	Self attention block	Multi head scaled dot product attention with LayerNorm.
crf.py	Linear chain CRF	Forward backward and Viterbi with hashed features.
word2vec.py	Word2Vec skip gram	Negative sampling, subsampling, unigram ^{0.75} noise.
dual_encoder.py	Bi encoder	Two MLP towers trained with in batch InfoNCE.
bpe.py	BPE tokenizer	Learns merges from the FAQ corpus.

5.2 Two real trainable neural models in PyTorch

Both of these were trained on our own data. There are no downloaded weights. Each layer is written from first principles, so a reader can step through every line of the forward pass.

Transformer intent classifier in api/app/myml/torch_transformer.py:

- Embedding layer that converts vocab ids to 64 dimensional vectors.
- Sinusoidal positional encoding added to the embedding.
- Two encoder blocks. Each block has multi head self attention with 4 heads, then a position wise feed forward with 128 hidden units, with LayerNorm and residual connections around both sub layers.
- Masked mean pool over the non padding token vectors.
- Linear head with softmax over 13 intent classes.
- Trained with AdamW, cosine learning rate, gradient clip, cross entropy loss.
- Auto detects CUDA. On CPU it trains 40 epochs in about 3 minutes. On a single GPU it trains in seconds.
- Final training loss 0.0002, training accuracy 1.000 on 1517 examples.

Encoder decoder seq2seq in api/app/myml/seq2seq.py:

- Bidirectional LSTM encoder. The forward and backward hidden states are summed and projected, so the decoder gets a single initial state.
- Bahdanau attention written out as three Linear layers (W_q , W_k , v) with a tanh non linearity. The attention runs over every encoder output at every decoder step.
- LSTM decoder cell. The input to the decoder at each step is the previous output token embedding concatenated with the attention context.
- Output head is a Linear layer over (decoder hidden, attention context) into the full vocabulary.
- Loss is teacher forced cross entropy. The fraction of true tokens vs predicted tokens fed back is a curriculum that starts at 1.0 and decays to 0.5.
- Training uses AdamW with cosine schedule and gradient clip.
- Greedy decoding at inference. Start with the BOS token, emit one token at a time until EOS or the maximum length is hit.
- Auto detects CUDA. The user can train the full corpus on a GPU in seconds.

6. How the parts connect

The simplest way to see how the project works is to follow a user message through the system.

```
User types in the browser
-> POST /chat to the FastAPI backend
-> retrieval.retrieve_answer()
    |--- analyze.py runs every NLP module
    |--- index.score(query) on TF-IDF, BM25, LSA, PPMI
    |--- index.top_k_with_prf(query)
    |--- info = best_match(scores)
    |--- generator.grounded_generate(query, faq_answer)
    |       |--- seq2seq.generate(query)
    |       |--- cosine sim against retrieved answer
    |       |--- fall back if too different
    |--- dialog manager wraps the result
    |--- auto_faq.enqueue_unmatched() if score < 0.45
    |--- _bump_faq_count() increments Mongo counter
-> JSON envelope back to the browser
-> Next.js renders the answer + NLP debug panel
```

The auto FAQ pipeline runs in the same process. When the queue has new entries, it is clustered every 5 turns. Clusters of size 3 or more become candidate FAQs. They go into the admin dashboard for review. Once approved they appear in the public FAQ list at /faq/<specialty>.

7. How the models are trained

7.1 Train the transformer intent classifier

Run this from the api folder. The script auto detects CUDA and falls back to CPU.

```
cd api
python train_transformer.py --verbose

# arguments:
#   --epochs 60      number of training epochs
#   --verbose        print loss + accuracy every 5 epochs
#   --out PATH       save checkpoint to PATH
```

What happens: the script loads intents.jsonl plus intents_augmented.jsonl, deduplicates by (text, label), and trains a 2 block transformer on the combined set. It saves the trained weights to api/app/mymtl/checkpoints/torch_intent.pt

7.2 Train the seq2seq generator

The seq2seq is trained on FAQ question to FAQ answer pairs, with paraphrase augmentation.

```
cd api
python train_seq2seq.py --epochs 60 --verbose

# for a quick CPU smoke run:
python train_seq2seq.py --fast --epochs 15
```

The default model size is d_emb 64, d_hid 128, max source 24, max target 80. The fast preset shrinks the model so a smoke training finishes in about a minute. The full model trained on a single GPU finishes in well under a minute and produces much better text.

7.3 Train and use the from scratch CRF NER

The CRF is trained the first time the API needs it. It uses ner_examples.jsonl plus silver labels generated by the dictionary NER over every FAQ question. The CRF then handles future NER calls at api/app/services/crf_ner.py.

8. How the auto FAQ pipeline works

This is the original goal of the project. The pipeline closes the loop: chat traffic becomes new FAQs without human writing.

The flow has 5 steps:

- Step 1: every chat turn that scores below 0.45 on retrieval is added to a queue. The queue is held in memory, one queue per specialty.
- Step 2: after every 5 new entries the queue is clustered. The clusterer is K Means from our myml package.
- Step 3: clusters of size 3 or more are considered worth promoting.
- Step 4: the centroid closest question becomes the representative. The seq2seq writes a draft answer for it.
- Step 5: the candidate is written to MongoDB with approved set to false and auto_generated set to true. The admin can edit, approve, or reject it from the admin dashboard.

Endpoints that drive this:

```
GET /admin/mining/status
POST /admin/mining/run/<specialty>
GET /admin/candidates/<specialty>
POST /admin/approve/<faq_id>
POST /admin/reject/<faq_id>
POST /admin/edit/<faq_id>
```

9. How to run the project

The whole stack can be brought up with one command if Docker is installed.

```
docker compose up --build

# After it finishes building:
# http://localhost:3000          web app
# http://localhost:8000/docs     FastAPI swagger
# http://localhost:8000/health   liveness probe
```

docker-compose.yml starts five services: web (Next.js), api (FastAPI plus models), mongo (FAQ store), qdrant (reserved for future vector search), and redis (reserved for the pipeline scheduler).

On the first boot the API checks if MongoDB has any FAQs. If the collection is empty, it auto seeds 214 FAQs from data/faqs.jsonl. After that point the counts and any new auto generated FAQs live in Mongo and survive restarts.

9.1 Running without Docker

```
# Backend
cd api
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
$env:FAQ_FORCE_MEMORY = "1"
python -m uvicorn app.main:app --port 8000
```

```
# Frontend (separate terminal)
cd web
npm install
npm run dev
```

10. How we measured the project

Two test scripts give the project's health a number.

10.1 final_test.py

This script hits the running API and the running Next.js frontend with 60 plus checks. It covers chat, faq, admin, stats, nlp endpoints, mining endpoints, and the static frontend routes.

```
python api/final_test.py      # 62 / 62 PASS
```

10.2 eval.py

This script runs every line of eval_queries.jsonl through the chat endpoint and computes retrieval metrics.

```
python api/eval.py           # 156 queries
```

Precision at 1	0.926
----------------	-------

MRR	0.940
-----	-------

Recall at 5	0.955
-------------	-------

Median latency	53 ms
----------------	-------

Cardiovascular P@1	0.917
--------------------	-------

Physiotherapy P@1	0.980
-------------------	-------

Radiology P@1	0.891
---------------	-------

Transformer accuracy	1.000
----------------------	-------

Seq2seq loss after smoke run	5.80 (CPU). On a GPU it reaches 0.05 in seconds.
------------------------------	--

11. Strengths and limits

Strengths

- No LLM. The project is reproducible and auditable and never depends on a third party model.
- Every model is ours. The reader can step through every line of the forward and backward pass.
- Hybrid retrieval gives 0.93 Precision at 1 on 156 queries.
- Generation is grounded. The seq2seq cannot write content the retrieved FAQ does not support.
- Auto FAQ mining closes the loop from chat traffic to clinician approved FAQs.
- Multi turn dialog. Coreference, slot filling, triage banners, empathic openers, disambiguation cards.
- Pakistan friendly. Roman Urdu mapping, 1122 emergency banner, PHI scrub at ingest.
- One command deploy with Docker. Mongo auto seeds itself.
- Real time analytics. The admin dashboard shows live FAQ counters.

Limits

- Small corpus. The 214 hand written FAQs cover the most common questions, but out of distribution questions still fall through to the best guess bucket.
- Eval keyword check is approximate. A correct answer that uses a slightly different word can be flagged as wrong by the keyword based eval script.
- Transformer training is small. 1517 examples after augmentation. It is enough to learn the 13 intents but a much bigger dataset would help generalisation.
- Seq2seq needs GPU training to be useful. On CPU it does not converge well within a reasonable time, so the grounding check falls back to retrieval most of the time.
- Roman Urdu is heuristic. About 80 word dictionary. A real transliteration would need a parallel corpus.
- Voice input is browser native. It works in Chrome and Edge only.

12. Future direction

- Grow the FAQ corpus to 500 or more with clinician validation.
- Train the seq2seq on a real GPU for many more epochs and emit better text by default.
- Add real Urdu transliteration with a parallel corpus.
- Replace the keyword based eval with a clinician rated NDCG metric.
- Add an active learning UI: low confidence chat turns get queued for clinician review and approved Q and A pairs feed back into the corpus.
- Add audit and rollback for the FAQ approval workflow.

13. Quick file index

Path	Kind	What it does
api/app/main.py	FastAPI app	Mounts all routers.
api/app/db.py	Storage	Mongo client plus in memory fallback. Auto seeds Mongo from JSONL.
api/app/services/retrieval.py	Core	Runs the 10 step pipeline for one chat turn.
api/app/services/generator.py	Service	Runs the seq2seq, grounds it, picks generated or retrieval text.
api/app/services/auto_faq.py	Service	Queue, K Means clusterer, candidate FAQ writer.
api/app/services/analyzer.py	Service	Runs every NLP module on the user text.
api/app/myml/*.py	Models	Every from scratch ML model in numpy or PyTorch.
api/app/nlp/*.py	NLP	Lemmatiser, POS, NER, sentiment, triage, intent, dialog, etc.
api/app/routers/*.py	HTTP	chat, faq, admin, nlp, stats endpoints.
api/train_transformer.py	Trainer	CLI to train the intent classifier.
api/train_seq2seq.py	Trainer	CLI to train the seq2seq generator.
api/eval.py	Eval	Precision at 1, MRR, Recall at 5 over the held out queries.
api/final_test.py	Test	Full integration suite against the live API plus Next.js routes.
data/build_jsonl.py	Data	Regenerates every JSONL from the curated sources.
data/augment_intents.py	Data	6 way augmentation of intents.
data/expand_corpus_v[1-3].py	Data	Three rounds of FAQ and intent and eval expansion.
data/synth_chat_logs.py	Data	Generates the 14k synthetic chat sessions.
web/src/app/page.tsx	UI	Landing page.
web/src/app/chat/[specialty]/page.tsx	UI	Chat page.
web/src/app/playground/page.tsx	UI	NLP playground.
web/src/app/architecture/page.tsx	UI	Architecture diagram and write up.
web/src/app/admin/page.tsx	UI	Admin dashboard with live counts.
docker-compose.yml	Deploy	Brings up web, api, mongo, qdrant, redis.